A COMPARATIVE STUDY OF DATABASE TECHNOLOGIES FOR TIME SERIES DATA

Yosafe Fesaha Oqbamecail

Master's Thesis in Applied Computer Science and Engineering

Engineering Data Science Specialisation

Faculty of Technology, Environmental and Social Sciences

Western Norway University of Applied Sciences

24th March 2026

ABSTRACT

This is where you write the abstract.

ACKNOWLEDGMENTS

I would like to thank . . .

Contents

1 Introduction **9**
1.1 Background of the Study . . . 10
1.1.1 The SFI Smart Ocean Platform . . . 10
1.1.2 Motivation . . . 11
1.2 Problem Statement . . . 11
1.3 Research Questions . . . 12
1.4 Research Methodology . . . 12
1.4.1 Use of AI Tools . . . 14
1.5 Thesis Outline . . . 14

2 Background and Related Work **17**
2.1 Time Series Data and IoT Workloads . . . 18
2.1.1 Definitions of Time Series Data . . . 18
2.1.2 Characteristics of Sensor-Generated Time Series Data . . . 18
2.1.3 Identified Challenges in IoT Time Series Management . . . 19
2.1.4 Scalability and Ingestion Constraints . . . 20
2.2 Database Architectures for Time Series Data . . . 20
2.2.1 Relational Databases and Time-Based Extensions . . . 21
2.2.2 Native Time Series Database Systems . . . 22
2.2.3 Column-Oriented Analytical Databases . . . 22
2.2.4 NoSQL and Document-Oriented Approaches . . . 23
2.2.5 Monitoring and Metrics-Focused Systems . . . 23
2.3 Core Properties of Time Series Database Systems . . . 24
2.4 Data Modeling and Indexing Strategies for Time Series Databases . . . 25
2.4.1 Time Series Data Models . . . 25
2.4.2 Indexing Strategies for Timeseries Data . . . 26
2.4.3 Storage Layout and Partitioning Strategies . . . 27
2.4.4 Compression Techniques for Time Series Data . . . 28
2.4.5 Query Optimization in Time Series Databases . . . 29
2.5 Performance Evaluation of Database Systems . . . 30
2.5.1 Principles of Database Performance Evaluation . . . 30
2.5.2 Workload Modeling for Time Series Systems . . . 31
2.5.3 Evaluation Metrics in Time Series Studies . . . 31
2.5.4 Limitations in Existing Comparative Studies . . . 32

3 Design and Analysis **33**

3.1 Design and Analysis . . . 34
3.2 Selection of Database Systems . . . 35
3.3 Dataset Design and Characteristics . . . 37
3.4 Workload Design . . . 37
3.4.1 Ingestion Workload . . . 38
3.4.2 Query Workload . . . 38
3.4.3 Query Execution Approaches . . . 39
3.4.4 Workload Consistency and Control . . . 39
3.5 Experimental Environment and System Configuration . . . 39
3.5.1 Hardware Environment . . . 39
3.5.2 Software Environment . . . 40
3.5.3 Database Configuration . . . 40
3.5.4 Execution Control and Isolation . . . 40
3.5.5 Considerations for Reproducibility . . . 41
3.6 Evaluation Metrics . . . 41
3.6.1 Ingestion Throughput . . . 41
3.6.2 Query Latency . . . 41
3.6.3 Storage Efficiency . . . 42
3.6.4 Scalability . . . 42
3.6.5 Resource Utilization . . . 42
3.7 Data Analysis Methods . . . 42
3.7.1 Repetition and Averaging . . . 43
3.7.2 Normalization of Results . . . 43
3.7.3 Comparative Analysis . . . 43
3.7.4 Trend Analysis . . . 43
3.7.5 Handling Variability and Outliers . . . 44
3.7.6 Experimental Procedure . . . 44
3.8 Threats to Validity and Design Limitations . . . 44
3.9 Synthesis of Design Decisions . . . 45

4 Implementation and Prototypes **47**

5 Evaluation and Results **49**

6 Conclusions and Future Work **51**
6.1 Main Contributions . . . 51
6.2 Conclusions on Research Questions . . . 51
6.3 Threats to Validity . . . 51
6.4 Future Work . . . 51

Bibliography **52**

A Source Code **57**

B Research Data **59**

CHAPTER 1

INTRODUCTION

It is difficult to discuss global ecosystems without acknowledging the role oceans and seas play especially in climate regulation, energy production and food security (Mendler de Suarez et al. [14]). Monitoring such large and dynamic environments needs advanced communication and sensing technologies. Underwater Sensor Networks (UWSNs) have come up as a solution for continuous and real-time data collection in subsea infrastructure, marine ecosystems and oceanographic research (Domingos et al. [5]).

The SFI Smart Ocean project is focused on creating an autonomous and flexible marine observation platform that integrates cloud-based management systems with underwater sensor networks (Forskningsrådet [6]). This platform allows long-term, large-scale monitoring of underwater environments and industrial installations. The multi-parameter sensor data is collected at multiple underwater sensor sites and transmitted to surface nodes and cloud infrastructure where it is stored, processed, and analyzed.

However, underwater deployments introduce additional constraints when compared to terrestrial deployments. Acoustic communication has low bandwidth, high latency, and low reliability; and the sensors operate in extreme temperature, pressure, biofouling, corrosion and long-term electronic drift conditions. These constraints will require robust and fault-tolerant data management systems once the data arrives at land-based infrastructure.

One of the main technical challenges comes from the volume and speed of time series data generated by the sensor networks. There are typically hundreds of sensors continuously transmitting time-stamped observation across many variables per device. With many deployments, this creates thousands of datasets which require efficiency to ingest and analyze when needed, store in the long-term and access in real-time.

Traditional relational database management systems (RDBMS) offer strong consistency and mature tooling, but they are not inherently optimized for high-throughput time series workloads. In response to the growth of IoT and sensor-based systems, specialized Time Series Management Systems (TSMSs) have

emerged. These systems are designed to efficiently ingest, index, compress, and query time-stamped data streams, often offering built-in functions for time window aggregation, down-sampling, and retention management (Jensen et al. [12]).

Because there are numerous database technologies, finding the right one with the right trade-offs for large scale marine sensor networks can take time. Therefore, there are no commonly accepted best models for databases regarding tradeoffs of ingestion performance, query execution time, scalability, storage efficiency, and complexity to operate. Many benchmarks for databases currently exist in the form of general use "IoT" databases or databases for financial data streams, which do not usually represent the true workloads of underwater sensor platforms.

This thesis fills this gap by providing a comparative evaluation of selected database technologies for large-scale time series data in the Smart Ocean. It evaluates various types of databases using a structured evaluation approach assessing ingestion rates, query performance, scalability act, and efficiency of storage and present the results as empirical evidence. The results of this study will provide valuable comparisons of the performance as well as identify architectural trade-offs which will help in making informed decisions on selecting a specific database for the Smart Ocean and similar sensor-driven systems.

1.1 Background of the Study

1.1.1 The SFI Smart Ocean Platform

The SFI Smart Ocean project, is a project funded by the Norwegian Research Council. The collaborative initiative involves research institutions and industry partners working together to develop intelligent and scalable marine monitoring systems (SFI Smart Ocean [22]). The project is aimed at creating an autonomous Smart Ocean platform capable of supporting long-term underwater observation and industrial marine operations. Its research efforts span three interconnected domains.

First, it aims to deploy advanced underwater measurement and sensor technologies that can monitor the harsh subsea environment continuously. The sensors are engineered to use energy efficiently and for a long time. This means that they can collect multi-parameter data such as pressure, salinity, temperature and other environmental variables. Secondly, the project aims to solve the problem with underwater wireless sensor networks based on acoustic communication. Acoustic transmission has physical limitations, meaning that the networks have to operate under constrained bandwidth, intermittent connectivity and high latency. Due to this, data transfer becomes a major engineering challenge. Thirdly, the initiative is creating a cloud-based Smart Ocean platform that can ingest, store, process and visualize the data received from distributed

sensor nodes. This platform can integrate heterogeneous data streams using standardized APIs and data formats while making sure that the data is accessible in the long term, available when needed and scalable. After the sensor observations make it to the surface, they are forwarded to a centralized infrastructure that is designed for reliability and efficiency.

1.1.2 *Motivation*

Once the Smart Ocean's underwater network is deployed, it will generate continuous, time series data streams. Every sensor will be able to produce timestamped measurements across different variables, possibly. As more and more sensors are introduced, the data will grow and will need long-term storage. This workload comes with data management challenges like efficient indexing of the time-stamped data, requirements for high ingestion throughput, need for long-term storage and need for real-time query support so that the data can be useful for monitoring, alerting and supporting analytical queries. The data will also need to be scalable across a growing number of sensor deployments.

Traditionally, relational database management systems (RDBMS) would provide mature transaction support and reliability but they were not designed for the storage and querying demands of time series data. Characteristically, time series workloads require sequential writes, aggregation over windows, time-based filtering and retention management which need specialized indexing and storage strategies. Dedicated time series databases and alternative architectural approaches have come up to meet these demands. These systems use techniques such as write-optimized storage engines, time-partitioned storage, columnar compression and pre-aggregation mechanisms to better the performance of time-centric workloads (Jensen et al. [12]).

However, it is yet to be established whether these database technologies are suitable for large-scale underwater sensor networks especially in marine deployments. The Smart Ocean platform needs high ingestion performance, scalability, reliability, storage efficiency and operational maintainability over long deployment periods. Picking an appropriate database technology is therefore a critical architectural decision. A systematic comparative evaluation of candidate systems is necessary to determine which technologies provide the most favorable performance and scalability trade-offs under representative Smart Ocean workloads.

1.2 Problem Statement

The Smart Ocean platform is expected to manage continuous large amounts of time-stamped data generated by a wide network of underwater sensors. In most cases, these data streams are append-heavy, they require time-based indexing and long-term storage and they have mixed workloads that involve analytical processing of historical data and real-time monitoring queries. Due to all these

considerations, selecting the appropriate database technology is a major architectural challenge. Different database systems use different indexing strategies and storage models including but not limited to time-partitioned hyper tables, relational row-based storage and document-oriented storage. These differences affect ingestion throughput, storage efficiency, operational complexity, query latency and scalability.

Even though many database technologies claim to work with time series workloads, the way they perform specially under large-scale marine sensor deployments is yet to be properly evaluated. Existing benchmarks focus on generic IoT workloads, monitoring metrics or financial trading streams which may not accurately reflect query behavior, ingestion patterns and the long-term storage demand of underwater sensor networks. This thesis looks into how different database technologies behave under Smart Ocean time series workloads. In this case, the Smart Ocean platform represents other similar data types. The objective is not to find a universally optimal database system but to evaluate and compare how the existing ones perform and the scalability trade-offs one has to make with the selected technologies within the defined operational context.

1.3 Research Questions

This research will seek to establish the database technology that provides the most favorable performance and scalability trade-offs for dealing with large-scale time series data generated from the Smart Ocean underwater sensor network. It will specifically seek to answer the following questions:

- **Research Question 1:** *How do the architectural designs of timeseries database influence their performance characteristics for time series workloads?*
- **Research Question 2:** *How do state-of-the-art time series database systems perform with respect to ingestion throughput, query performance, scalability, and storage efficiency?*

1.4 Research Methodology

To systematically evaluate database technologies for the Smart Ocean time series workload, this thesis adopts the technology evaluation methodology proposed by Brown and Wallnau [3]. This framework provides a structured approach for assessing competing technologies by identifying measurable performance deltas under representative operational conditions. The methodology consists of three sequential phases: descriptive modeling, experiment design, and experiment evaluation. Figure 1.1 illustrates the overall structure of this framework.

The descriptive modeling phase establishes the relationship between the problem domain and candidate technologies. In this thesis, the problem domain is defined by the Smart Ocean time series workload, characterized by high ingestion rates, append-heavy data patterns, time-based queries, long-term reten-

tion requirements, and scalability demands. During this phase, the workload characteristics and system requirements are formally defined. These include expected ingestion throughput, query types (e.g., time-window aggregation, range scans, real-time monitoring queries), data retention duration, storage efficiency considerations, and scalability constraints. Simultaneously, the following candidate database technologies are analyzed in terms of their architectural principles: TimescaleDB, a time-series database built as an extension of PostgreSQL, InfluxDB, a purpose-built time-series database, and MongoDB, a general-purpose NoSQL document database. The analysis includes examining their storage models, indexing strategies, compression mechanisms, clustering capabilities, and time series support features. The goal of this phase is to "situate" each technology within the Smart Ocean problem domain and identify the relevant characteristics that influence performance outcomes.

The experiment design phase translates the descriptive analysis into measurable hypotheses and benchmark configurations. Based on the workload requirements identified in the previous phase, hypotheses are formulated regarding expected system behavior. For example, column-oriented storage engines may demonstrate improved compression and analytical query performance, while specialized time series engines may exhibit superior ingestion throughput. Representative workload models are then constructed to simulate Smart Ocean data streams. These workloads emulate realistic sensor behavior, including configurable sampling rates, multi-variable measurements, and long-term data accumulation. A standardized benchmarking environment is defined to ensure fair comparison across systems, including consistent hardware configuration, controlled system tuning, and uniform dataset characteristics. Performance metrics are selected to evaluate system behavior. These include ingestion throughput, query latency, storage efficiency, scalability under increasing data volumes, and resource utilization. Clear evaluation criteria are established to measure performance deltas between candidate technologies.

In the experiment evaluation phase, the selected database systems are deployed within the defined benchmarking environment and tested under the representative workloads. Experimental results are collected and analyzed according to the predefined metrics. The evaluation includes quantitative performance benchmarking as well as compatibility assessment with Smart Ocean requirements. Observed performance deltas are interpreted in relation to architectural differences identified in the descriptive modeling phase. This structured comparison enables a systematic technology assessment rather than an isolated performance ranking. The outcome of this phase is a comparative evaluation of the candidate database technologies, highlighting their strengths, limitations, and trade-offs in the context of large-scale marine time series data management.

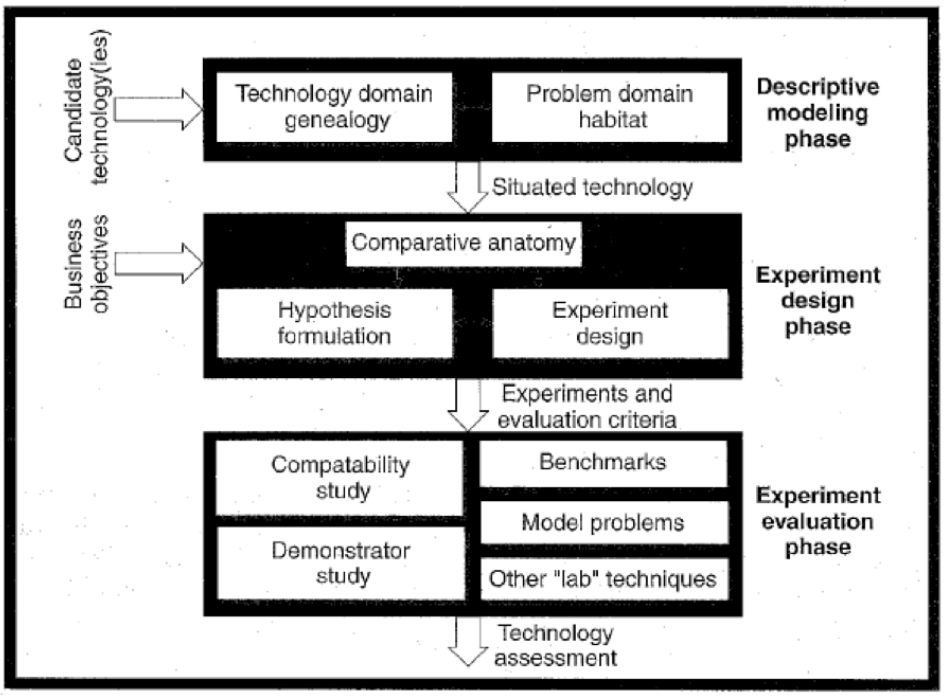

Fig. 1.1: Software technology evaluation framework (adapted from Brown and Wallnau [3]).

1.4.1 *Use of AI Tools*

Throughout the writing process of this thesis, AI tools were used in a limited and supportive capacity. Microsoft Copilot was used exclusively for correcting spelling errors and improving the clarity of the language. claude.ai was was used to help outline the structure of the thesis, explain concepts during the writing process, and assist with debugging code in the experimental framework. All findings, research decisions, and written content are independently produced by the author.

1.5 Thesis Outline

The rest of this thesis is organized as follows:

1. **Chapter 2 Background and Related Work:** Provides the necessary background and context. This chapter reviews the challenges and characteristics of time series data, and surveys existing database technologies for time series data management. It discusses related work and prior research around database performance in IoT and sensor data scenarios.

2. **Chapter 3 Design and Analysis:** Outlines the design of the experimental evaluation, including the selection of database technologies, the design of test workloads, and the metrics used for assessment.

3. **Chapter 4 Implementation and Prototypes:** Describes the implementation of the experimental setup, including the deployment of database systems and the creation of test environments. Details the development of proto-

types or models used for evaluation.

4. **Chapter 5 Evaluation and Results:** Presents the empirical results of the comparative evaluation of database technologies. Includes ingestion throughput, query performance, storage efficiency, and other findings.
5. **Chapter 6 Conclusion and Future Work:** Summarizes key findings and contributions, provides recommendations for the Smart Ocean platform, and suggests directions for future work.

CHAPTER 2

BACKGROUND AND RELATED WORK

The incremental and fast growth of sensor-driven systems and Internet of Things (IoT) infrastructures have by necessity changed the landscape of data management. Throughout scientific, financial, industrial and environmental domains, time-stamped data streams are continuously generated at a growing scale. As discussed in Chapter 1, large-scale monitoring platforms like the Smart Ocean system create sustained append-heavy time series data which needs reliable storage, long-term retention and efficient querying. These demands have led to the creation of specialized database architectures and methods for evaluation intended to accommodate high-ingestion and time-centric workloads.

Traditional transactional datasets and time series data differ in a number of critical respects. Time series data has a temporal ordering that is intrinsic to both query behavior and interpretation. Its generation patterns are often continuous and append-dominated instead of being update driven (Geurts [7]). The emergence of IoT systems has ramped up these characteristics, introducing multidimensional, high-velocity and high-cardinality data streams generated from distributed sensors (Gubbi et al. [10]). Consequently, conventional database architectures initially optimized for Online Transaction Processing (OLTP) workloads struggle structurally when applied to time-centric data management cases.

To address this, different architectural approaches have come up. Relational database systems now include time-based partitioning and indexing strategies to support temporal workloads. Native time series databases are designed specifically to optimize append-heavy time-range and ingestion queries. Analytical engines that are column-oriented now leverage compression and have vectorized execution improve the performance for large-scale aggregations. Document-oriented systems have included flexible schema models that can accommodate heterogeneous sensor data while cloud-managed time series platforms offer scalable infrastructural abstractions for IoT applications.

Alongside architectural innovation, methodologies for empirical evaluation have adapted to assess system performance under operational conditions that

are more realistic. Comparative performance studies have measured query latency, ingestion throughput, scalability behavior and storage efficiency across different database platforms. Workload modeling frameworks like YCSB (Cooper et al. [4]) have tried to standardize experimental conditions for IoT-oriented and cloud systems. However, as will be demonstrated in the subsequent sections, architectural coverage, variations in workload realism and methodological transparency complicate comparisons across studies.

Factoring in how diverse database paradigms are and the complexity of time series workloads, a review of the existing academic research is necessary so as to situate the present research within existing scholarship. This chapter looks at existing prior work done across four interrelated domains: database architectural paradigms for temporal data management, the characteristics of time series data and IoT workloads, methodological approaches to empirical system evaluation and comparative studies of time series database performance. Using this synthesis, recurring limitations and research gaps are identified. In the analysis, it is demonstrated that even though there has been significant progress to understand time series database performance, cross-architecture comparative evaluation under realistic workload conditions is yet to be well-explored. This gap motivates the investigation undertaken in the subsequent chapters.

2.1 Time Series Data and IoT Workloads

2.1.1 *Definitions of Time Series Data*

Time series data refers to a set of data points measured and recorded over time, where the order of the data points in time is critical for understanding and analyzing the data (Geurts [7]). Unlike cross-sectional data, time series data cannot be changed arbitrarily. In computer systems, time series data is typically modeled as a tuple containing a time component and one or more measurements.

Stonebraker, Çetintemel, and Zdonik [24] highlight the importance of the prevalence of continuous data arrival and window queries in stream and time-oriented databases, which differ from traditional transaction-oriented databases. This gives rise to a number of important implications for storage systems, where time series data is to be stored. Time series workloads, therefore, require temporal indexing, window aggregation, and sequential writes, which make it different from Online Transaction Processing (OLTP) workloads, which have characteristics like atomic transactions, complex joins, and high update frequency.

2.1.2 *Characteristics of Sensor-Generated Time Series Data*

IoT systems typically have interconnected sensors which continually collect and send out environmental or operational data (Gubbi et al. [10]). These sensors

produce structured high-frequency data streams which needs persistent storage and real-time accessibility. Existing literature has identified several defining characteristics of sensor-generated time series data.

Large-scale sensor networks are able to create millions of records per hour especially when they measure many parameters simultaneously (Zanella et al. [27]). As the number of sensor deployments increases, ingestion rates increase. Sensor data tends to be multidimensional. One time stamp could correspond to many attributes such as vibration, salinity, temperature, pressure and many other environmental indicators. This complicates schema design and strategies for indexing especially when different sensors measure heterogeneous variable sets.

Thirdly, IoT data is often highly cardinal. Cardinality refers to the number of distinct combinations of dimensions like geographic regions, sensor identifiers, measurement categories or device types. Datasets with high cardinality increase how complex the indexing needs to be as well as the memory overhead, presenting challenges for scalability (Bandyopadhyay and Sen [2]). Fourthly, sensor data often needs to be stored over a long period. Industrial deployments and environmental monitoring platforms operate continuously for years. Long-term data retention is necessary to allow historical trend analysis, compliance verification, anomaly detection and predictive modeling (Gubbi et al. [10]). To keep such large amounts of data imposes storage demands and pressure on query optimization mechanisms.

Finally, time series data generated by sensors tends to be append-only. Historical observations are rarely modified, and data updates primarily involve introducing new records. This append-heavy trait influences storage engine design, encouraging sequential write optimization and time-based partitioning.

2.1.3 *Identified Challenges in IoT Time Series Management*

The literature identifies many technical challenges associated with managing IoT-generated time series data. One of the major ones is storage scalability. Even with moderate sampling frequencies, one still ends up with billions of records over a long operational period (Gubbi et al. [10]). Storage systems have to support sustained ingestion rates while staying durable.

A second challenge involves query efficiency. Often, monitoring systems rely on time-based queries like retrieving measurements in specific time intervals or computing windowed aggregates. When indexing and partitioning strategies are not optimized, query latency can degrade significantly with growing datasets (Stonebraker et al. [24]). It does not help matters that continuous ingestion leads to unbounded data growth. Many IoT systems implement down sampling strategies, retention policies or hierarchical storage architectures which introduce another dynamic. Down sampling reduces the requirements for storage

by aggregating high-resolution data into coarser time intervals. For this to work, database systems have to be able to support efficient deletion, aggregation operations and compression without interrupting ingestion (Zanella et al. [27]).

Data consistency and reliability further complicate the management of time series data. Distributed sensor networks sometimes experience intermittent connectivity, delayed transmissions or packet loss. When this happens, time series systems have to be able to tolerate out-of-order data arrival and variable ingestion patterns without losing temporal integrity.

2.1.4 *Scalability and Ingestion Constraints*

In time series systems, scalability can be examined in both horizontal and vertical dimensions. Vertical scalability is about increasing computational resources inside a single node such as memory allocation, CPU capacity or disk throughput. In contrast, horizontal scalability involves distributing data across multiple nodes to accommodate higher storage volumes and ingestion rates. As discussed in an earlier section, high-ingestion environments represent a primary bottleneck in IoT deployments. Unlike read-dominated workloads, sensor-based systems tend to be write-intensive. Continuous insert operations overwhelm storage engines if indexing, logging, or synchronization mechanisms are not optimized for sequential access patterns (Stonebraker et al. [24]). Excessive indexing easily degrades write performance while insufficient indexing increases query latency.

Distributed scaling introduces other complexities, including coordination costs, data partitioning strategies, and replication overhead. A common way to distribute load has been partitioning by device identifier or time interval, but poorly chosen partitioning schemes can cause uneven resource utilization or hot spotting. Database systems have to be able to navigate trade-offs between storage compression, query responsiveness and ingestion efficiency. In large-scale marine monitoring systems and similar deployments, these ingestion and scalability constraints directly influence decisions for database selection. Any database technology evaluation for time series workloads has to consider realistic cardinality levels, ingestion rates, distributed deployment scenarios and retention durations.

2.2 Database Architectures for Time Series Data

The need to manage time series data has led to a number of different database architectures that are aimed at enabling temporal data processing at scale. Relational database systems remain dominant, but specialized time series, columnar analytical, and NoSQL databases have been developed to overcome their limitations in scalability and efficient storage for analytical purposes. The design includes different options regarding how the data should be organized, in-

dexed, and distributed over multiple structures. These design considerations need to be taken into account when assessing the database technologies for large sensor-based environments that require analytical as well as operational monitoring.

2.2.1 Relational Databases and Time-Based Extensions

Relational database management systems (RDBMS), notably PostgreSQL and MySQL, have traditionally excelled in handling structured data in various fields, particularly in enterprise and scientific contexts. These rely on row-oriented storage models, strong transactional guarantees, and structured query language (SQL) as the main interface for data manipulation and retrieval. Their mature ecosystems offer tooling, documentation, and operational best practices. However, relational database architectures were initially designed for online transaction processing (OLTP) environments, where workloads are composed of comparatively small transactions and involve a high volume of updates and deletes as well as point lookups.

On the other hand, temporal datasets accumulate continuously over time and are often looked up over bounded intervals instead of individual entries. Since the row-oriented storage engines' design goal is to optimize access to entire record sets, there may be performance restrictions when a query requires scanning a vast number of historical data or making aggregates of data for a given time interval. Correspondingly, structural optimizations were created in modern relational systems to advance their appropriateness with temporal datasets. One of such approaches is to partition tables based on time boundaries. Here, query scanning is restricted to the relevant portions of the dataset, to reduce index sizes and enhance efficient query execution. For instance, there are declarative partitions in PostgreSQL for dividing tables automatically across specified temporal boundaries, with queries being directed to access specific partitions rather than the whole dataset.

Further refinements are brought in via extensions such as TimescaleDB (Timescale, Inc. [26]). TimescaleDB is built on top of PostgreSQL and has introduced hyper tables that are abstracted over both the temporal and optional spatial dimensions. This allows for datasets to be distributed in a transparent manner across multiple partitions while maintaining the relational model, and SQL compatibility. Consequently, the scalability benefit does not depend on changing the query interface since the execution plan for temporal queries can be optimized and streamlined. However, given the potentially massive historical archives, performance limitations may remain in the relational systems. This implies that large aggregation queries need to scan entire partitions, even when only a subset of attributes is of interest (Lamb et al. [13]). Relational databases work well because of their generality and extensive support infrastructure, but alternative designs are suggested for more efficient analytical workloads.

2.2.2 *Native Time Series Database Systems*

Native time series databases (TSDBs) represent a class of systems designed specifically to manage chronologically indexed datasets. Unlike general-purpose relational databases, these systems adopt storage and indexing mechanisms that align directly with the access patterns typical of temporal data. Many TSDB implementations employ log-structured storage engines in which new observations are appended sequentially to disk structures optimized for continuous data ingestion. This sequential organization reduces disk seek operations and enables efficient compression techniques that exploit the similarity between adjacent measurements.

InfluxDB is a great illustration for this design philosophy through its Time Structured Merge Tree (TSM) storage engine. There, data is initially written to memory buffers and then it is periodically compacted into immutable on-disk segments that are organized by time range (InfluxData, Inc. [11]).. This strategy lowers write amplification while supporting efficiency in retrieval within specified time windows (Nasar and Kausar [17]). In addition, retention policies enable administrators to automatically remove outdated data, which allows long-term deployments to operate without extensive manual oversight.

Specialized architectures introduce certain trade-offs. Some TSDB platforms provide limited relational query capabilities, particularly for operations that require multi-table joins or complex relational constraints. What is more, the surrounding ecosystem for specialized databases may be less mature than that of long-established relational systems, which can affect the availability of tooling and operational support.

2.2.3 *Column-Oriented Analytical Databases*

Another architectural paradigm that works for large-scale analytical workloads is column-oriented database systems. Different from row-based storage models, columns organize data by attribute and keep each column independent. This design makes several performance advantages for analytical processing possible. To start with, it allows every attribute to be compressed with encoding techniques tailored to the distribution of values inside the said column. Consequently, storage requirements get significantly reduced especially when datasets have repeated or slowly changing values. Secondly, query engines are able to load just the required columns for each computation reducing input/output overhead during execution.

ClickHouse is an example of a modern column-oriented database intended to support large scale data (Schulze et al. [21]). It executes using vectorized processing which evaluates operations in value batches simultaneously instead of processing records individually. This approach allows the system to rely on CPU cache locality and parallelism to improve query throughput for aggregation-intensive tasks. In temporal data analysis, majority of the queries

involve computing metrics like percentiles, averages or counts across long observation windows. Column-oriented systems are very well suited for this because they can scan and put together large volumes of data efficiently. For example, analytical queries examining sensor trends over a long monitoring period are often executed using fewer disk accesses compared to row-based systems.

Even so, column stores face performance challenges in environments where data is added continuously at extremely high rates. Maintaining columnar compression structures while making frequent updates can introduce overhead which affects ingestion performance. Due to this, column-oriented databases get deployed in architectures where analytical processing and operational ingestion are separated. Then, they become downstream analytical engines for historical datasets.

2.2.4 NoSQL and Document-Oriented Approaches

NoSQL databases were introduced to address scalability and inferior schema flexibility witnessed in traditional relational systems. Semi-structured data representations are often used, which can change in form after a given period, as opposed to mandatory relational schema provisions. Document-oriented databases as MongoDB store information in the form of JSON-like documents with the ability to set nested fields in each record or remove some attributes for one record (MongoDB, Inc. [16]). This is crucial in situations where devices or sources measure different things, and the type of measurement data can change as new measuring devices are added.

For chronologically indexed datasets, MongoDB has recently formalized specialized collections. The collections create observations stored in internal buckets that contain related measurements. This is achieved by reducing the metadata requirements and limiting index modifications, thus enhancing storage optimization and write efficiency for chronologically indexed datasets (MongoDB Documentation Team [15]). In addition, document databases implement distributed scaling with sharding, where datasets are divided into partitions within groups of nodes in a cluster. Sharding makes it possible to scale to bigger data sizes and workloads by spreading the data and the computation across multiple machines.

Despite these advantages, NoSQL systems generally adopt less strict consistency models compared to relational databases. Taipalus [25] notes that while RDBMSs enforce strong ACID consistency, NoSQL systems typically relax these guarantees to achieve higher performance and scalability for workloads with large numbers of concurrent users.

2.2.5 Monitoring and Metrics-Focused Systems

Monitoring platforms like Prometheus [20] and Graphite [8] represent a specialized category of systems designed primarily for infrastructure observability

and operational metrics collection. These systems prioritize simplicity, reliability, and real-time alerting capabilities.

Prometheus uses a pull-based model of data collection where monitored services expose endpoint metrics which are periodically scraped by the monitoring server [20]. The data collected is then stored in a local time series storage engine that is optimized for rapid retrieval of recent data [19]. This architecture makes it possible to evaluate alerting rules and dashboard queries fast in operational monitoring environments.

Graphite, on the other hand, uses a different design centered around hierarchical metric naming conventions [8]. The metrics get stored in fixed-size files in formats that maintain pre-aggregated values at many resolutions [8]. This structure simplifies long-term trend visualization but it also introduces constraints on query flexibility since the data has already been aggregated during storage.

Systems made for monitoring environments often emphasize predictable performance for alert evaluation instead of general-purpose analytical exploration. Consequently, their applicability to complex sensor analytics especially those using multi-dimensional datasets or needed extended historical analysis, has to be assessed carefully. These systems offer valuable capabilities for operational visibility but it remains to be seen whether they are suitable for large-scale scientific data analysis.

2.3 Core Properties of Time Series Database Systems

The preceding discussion of time series workloads and database architectures indicates that systems designed for temporal data management tend to share a number of recurring architectural properties. Although implementations vary across relational extensions, native time series engines, column-oriented databases, document-oriented platforms, and managed cloud services, the literature suggests that effective time series database systems exhibit several core characteristics shaped by the structural demands of append-heavy, time-ordered data.

First, append-optimized storage is fundamental. Time series workloads require a lot of sequential insert operations rather than transactional updates, as noted by [12]. Because of this, efficient time series systems prioritize high-throughput write paths, frequently employing log-structured storage mechanisms or time-partitioned tables to reduce write amplification and limit random disk access.

Second, time-based partitioning is necessary for both performance and scalability. Dividing data into temporal segments reduces index size, improves the efficiency of time-range queries, and simplifies retention management. Relational extensions like hyper tables, as well as native time series engines, rely on time-aware segmentation to optimize ingestion and retrieval operations.

Third, efficient execution of range queries and windowed aggregations is essential. Time series analysis commonly involves retrieving measurements within

bounded intervals and computing statistical summaries [12]. Database architectures must therefore support rapid scanning of contiguous temporal segments, whether through row-based indexing strategies or column-oriented compression and vectorized execution [12].

Fourth, compression made for temporal locality has a major impact on storage efficiency. Because adjacent time series values often have strong correlation, specialized encoding schemes can substantially reduce storage overhead without lowering query performance. Column-oriented systems, specifically, benefit from homogeneous storage structures that support high compression ratios (Sokolinsky [23]).

Finally, both vertical and horizontal scalability remains a defining property. As discussed in previous sections, ingestion rates and dataset size increase proportionally with deployment scale. Systems must therefore sustain growing write loads and query demands through efficient resource utilization and, where applicable, distributed data partitioning.

These core properties do not prescribe a single architectural solution; rather, they establish criteria by which time series database systems can be evaluated. Differences in how individual systems implement append optimization, partitioning, compression, and scalability mechanisms provide the basis for meaningful comparative analysis. These core properties form the foundation for the experiment.

2.4 Data Modeling and Indexing Strategies for Time Series Databases

2.4.1 *Time Series Data Models*

Efficient time series data management not only depends on the database architecture but also on the way the data is modeled within the storage system. A time series data model defines the logical structure used to represent, retrieve and store temporal observations. It is different from traditional relational datasets in that time series data emphasizes chronology and continuous accumulation.

A common way to represent time series data is through the timestamp-value model, where each record contains a timestamp and one or more related measurements. In sensor-based environments, additional contextual attributes are often needed to explain the source of the observation. Many systems, therefore, use an extended timestamp-dimension-value model, in which dimensions represent metadata like sensor identifiers, device types, geographic locations, or measurement categories (Stonebraker et al. [24]). This format lets systems organize observations by both time and contextual attributes.

In practice, time series data models are created using either narrow schemas or

wide schemas. Narrow schemas store each measurement as a separate record with a timestamp, a measurement identifier, and a value. This structure allows for flexible schema changes and makes indexing easier across different data streams. Wide schemas, on the other hand, store multiple measurements linked to a single timestamp within the same record. This method reduces storage use and can improve query performance when retrieving multiple attributes at once.

The decision between narrow and wide schemas involves balancing storage efficiency, schema flexibility, and query performance. Narrow schemas are often preferred in systems that handle various sensor measurements, where different devices may produce different variables over time. However, wide schemas can simplify indexing when measurements have a consistent structure.

Modern time series databases often mix these modeling strategies with extra metadata structures called tags or dimensions. This combination enables efficient filtering across large datasets. As the number of devices being monitored grows, effective modeling of these dimensions becomes vital for maintaining query performance and managing high-cardinality datasets in IoT environments (Gubbi et al. [10]).

2.4.2 *Indexing Strategies for Timeseries Data*

For fast time series data retrieval and analysis, efficient indexing is necessary. Since these datasets grow to billions of records over a long period, indexing strategies have to support efficient query execution and high ingestion throughput. Different from traditional workloads where queries often target individual records, time series workloads include range queries, window aggregations and filtering by metadata (Stonebraker et al. [24]).

Fundamentally, time series systems use time-based indexing. Because many observations are ordered by time, indexing structures are designed to mirror them and allow efficient retrieval within specified temporal intervals. Many systems keep indexes on timestamp columns to speed up range scans and enable querying that retrieves data within defined windows of time. In relational extensions like TimescaleDB, this functionality gets used through composite indexes which combine timestamps and metadata attributes like device identifiers.

It is common to filter by device or measurement in large-scale sensor environments. Because of this, time series databases often embrace multi-dimensional indexing strategies which combine categorical and temporal attributes. They allow queries to efficiently retrieve data associated with specific sensors or geographies while still keeping it chronological. Some specialized databases even employ alternative mechanisms for indexing meant for high-cardinality datasets to reduce overhead while supporting efficiency.

Large-scale analytical systems also use skip indexes or sparce indexing structures to allow query engines to rule out irrelevant data blocks during scanning

operations. Column-oriented databases often rely on these mechanisms to improve query performance across many datasets (Abadi et al. [1]). Even though they have many performance advantages, indexing structures introduce tradeoffs. Maintaining complex indexes can lower write performance especially in append-heavy environments with high ingestion rates. Time series systems have to carefully balance indexing complexity against efficiency. Many platforms end up limiting indexing to small sets of frequently queried attributes while relying on compression techniques and sequential scanning for broader analytical workloads.

2.4.3 Storage Layout and Partitioning Strategies

Many time series database engines organize data into continuous storage segments instead of storing it as independent records. This way, incoming data is grouped into blocks representing a bounded temporal range or a specific stream. Each block will have a sequence of time stamped data streams stored sequentially in disk structures or memory. The grouping helps to reduce random disk access and allow queries to process large portions of data through sequential scans.

Segment-based storage structures typically have metadata summaries describing what is contained in each block. These summaries include attributes like the maximum and minimum timestamp in each block, the number of observations stored as well as statistical summaries of different values. As one executes a query, the database engine examines the metadata before accessing the actual data. If a block's timestamp data is outside the query interval, then the entire block is skipped. This technique is called block punning and it reduces disk reads when scanning historical datasets

Segmented storage makes it possible to run compaction processes in the background. As new observations get appended to the system, the data is often organized into small segments which later merge into larger blocks through compaction. Compaction reorganizes stored data to improve its locality and remove redundancy. Since these operations happen asynchronously, they let systems optimize storage layout without interrupting query activity or ongoing ingestion.

Another advantage to segment-based layouts is how compatible they are with immutable storage structures. After a segment is written to completion, it is usually treated as read-only. These segments make concurrency control simpler because multiple queries can get access to the same data without synchronization conflicts. They also allow database engines to run maintenance operations like reorganization on older segments while new data still arrives.

The relationship between query execution, segmented storage structures and ingestion buffers is shown in Figure 2.1. The diagram shows how incoming measurements get accumulated into sequential storage segments that are scan-

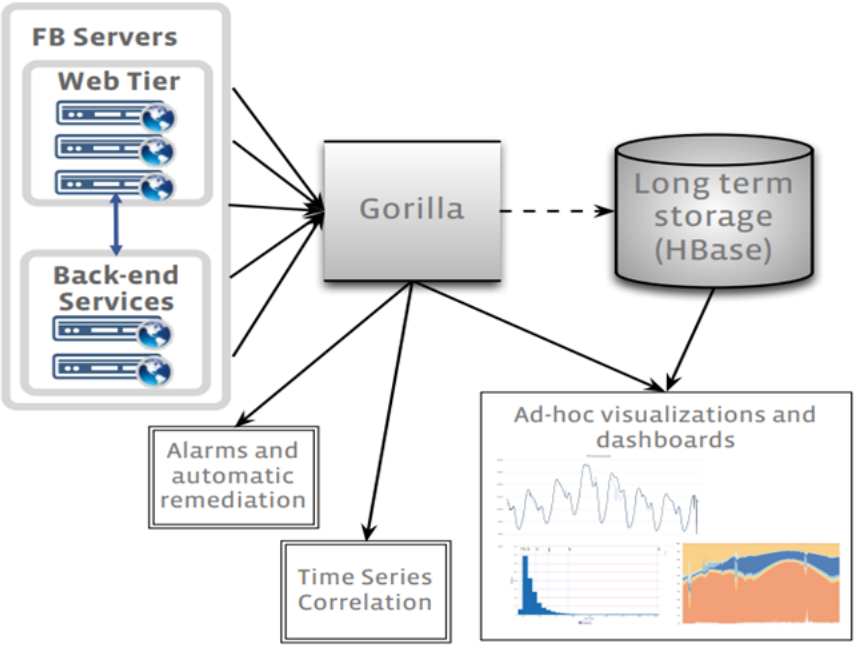

Fig. 2.1: Architecture of the Gorilla time-series monitoring database illustrating ingestion buffers and segmented storage structures used for efficient processing of temporal data. Adapted from (Pelkonen et al. [18]).

nable during analytical queries.

Segment-oriented storage layouts provide a structural mechanism for managing large volumes of chronologically ordered data.

2.4.4 *Compression Techniques for Time Series Data*

Generally, time series datasets exhibit strong temporal locality. Consecutive measurements will often change gradually instead of abruptly. This property makes it possible for database systems to apply specialized compression techniques to reduce storage requirements while keeping query performance efficient. A commonly used technique is delta encoding that stores the difference between successive values rather than storing every value on its own. In many datasets generated from sensor networks, measurements recorded at short intervals vary slightly from the previous ones. Encoding just the difference reduces the number of bits required per measurement. This method works perfectly for timestamps where time intervals between observations stay nearly constant.

An alternative approach used is run-length encoding (RLE) where sequences are compressed together if they are identical or have slowly changing values. RLE stores a single value for constant measurements over many sampling intervals along with a count representing the number of times it repeats. This technique is perfect in monitoring systems where metrics stay stable for long.

Modern time series databases frequently combine multiple compression tech-

niques within a single storage pipeline. One influential example is the compression algorithm introduced in Facebook's Gorilla monitoring database. The Gorilla system applies delta-of-delta encoding to timestamps and bit-level compression to measurement values, enabling highly efficient in-memory storage of large monitoring datasets. Pelkonen et al. [18] report that these techniques allow the system to achieve substantial reductions in memory consumption while still supporting fast retrieval of recent observations.

Compression also matters in column-oriented analytical databases because measurements inside one column often have similar statistical distributions. Compression algorithms exploit repeated values, predictable value ranges or small numeric differences to reduce storage overhead. In large-scale deployments, these compression strategies help manage long-term storage costs. It is worth noting that compression improves storage efficiency while introducing trade-offs. Decompression may become necessary during query execution especially when large historical datasets are involved.

2.4.5 *Query Optimization in Time Series Databases*

Time series analysis often involves operations that combine observations across temporal intervals instead of retrieving the records individually. Queries will typically compute moving averages, rolling statistics or summary metrics across sliding time windows. Since these operations often deal with large amounts of data, database systems have to use specialized query optimization strategies to keep performance acceptable. One way they do this is through window functions which allow queries to calculate over ordered data subsets defined by boundaries of time. Rather than scanning repeatedly, this execution plans process observations incrementally as new records enter the query window. This works for analytical operations like cumulative sums, moving averages and anomaly detection across different time intervals. Efficient execution requires that database engines maintain ordered data access paths and optimized pipelines for evaluating temporal aggregations (Stonebraker et al. [24]).

Another optimization strategy often used is the implementation of continuous queries. These queries automatically compute certain aggregations when new data arrives, updating the results incrementally instead of recomputing them from the whole dataset. Monitoring systems will often use this approach to keep up-to-date summaries of system metrics without running expensive analytical queries over and over. Continuous query mechanisms help reduce computational overhead for metrics that are frequently accessed.

Majority of time series databases support materialized rollups where historical data gets pre-aggregated into low-resolution summaries. Rather than storing just the raw observations, systems will periodically generate aggregated views that represent larger time intervals – like daily summaries. When queries request long-term historical trends, the database is able to retrieve results from summarized tables instead of scanning high-resolution raw data. This reduces

the query execution time while preserving the ability to analyze long-term data.

In large-scale monitoring environments, some systems will use approximate query processing techniques to speed up analytical queries over very large datasets. Approximations estimate aggregate values using sampled data subsets instead of evaluating the whole dataset. These estimates may sacrifice small degrees of precision, but they often provide better performance for exploratory analysis and tasks that require visualization.

2.5 Performance Evaluation of Database Systems

Evaluating database systems for time series workloads calls for systematic experimental design, controlled workload modeling, and carefully selected performance metrics. Unlike purely theoretical architectural analysis, empirical performance evaluation is concerned with quantifying how systems behave under representative operational conditions. In the context of large-scale sensor data management, such evaluation must consider ingestion throughput, query latency, scalability behavior, and storage efficiency while maintaining fairness across heterogeneous architectures. This section lays the theoretical framework for the benchmarking conducted in the next chapter.

2.5.1 Principles of Database Performance Evaluation

Historically, a lot of database research has focused on evaluation of database systems. Many researchers evaluated these systems using standardized performance metrics and workload models to make the results reproducible and comparable (Gray [9]). Early evaluation frameworks focused on transactional workloads but the growth of streaming and analytical systems has expanded the range of performance characteristics to consider.

Taipalus [25] emphasizes that meaningful evaluation demands a clear definition of not just workload characteristics, but also performance objectives and system configuration parameters. Without well-defined experimental conditions, performance comparisons run the risk of reflecting configuration bias instead of architectural differences. The time series systems, evaluation has to account for time-range queries, append-heavy writes and long-term retention.

Another basic principle involves transparency and repeatability. Experiments need to be designed in such a way that results are reproducible in similar conditions. This includes documenting things like storage configurations, hardware specifications, system tuning parameters and indexing strategies. Any differences in default configurations could influence how performance is observed especially across systems with distinct architectural paradigms. In heterogeneous environments like comparisons between relational extensions and native time series systems, careful control of the variables of the experiment is necessary. Each system may prioritize different strategies for optimization including distributed storage models, write buffering mechanisms or compression

algorithms.

2.5.2 Workload Modeling for Time Series Systems

Workload modeling is very critical in evaluating database performance. A workload represents the operations pattern executed against a database including query types, insert rates and data distributions. For time series systems, workloads have to reflect realistic injection patterns and query behavior typical of sensor and IoT deployments.

Cooper et al. [4] introduced the Yahoo! Cloud Serving Benchmark (YCSB) as a framework for evaluating NoSQL systems under configurable read-write workloads. Originally, it was designed for key-value stores but the concept of workload-driven evaluation has influenced broader research. In the context of time series data, workload models generally emphasize sustained insertion rates alongside periodic aggregation queries based on range.

Workload realism matters in sensor-based systems. Industrial and environmental monitoring platforms tend to produce data at regular intervals but network variability can introduce delayed arrivals or burstiness. A realistic workload model should consider these things. It must also consider data retention duration. Short-term evaluations may not show the performance degradation that happens as datasets grow over months or years. Scalable evaluation requires progressive data accumulation to echo long-lived deployments.

2.5.3 Evaluation Metrics in Time Series Studies

Performance evaluation of time series databases typically relies on several quantitative metrics:

- Ingestion throughput - measures the records inserted per second under sustained load. This metric is necessary for append-heavy sensor systems. High ingestion capacity ensures that incoming data streams can be stored without backlog or data loss.
- Query latency - measures the time required to run representative queries, like retrieving measurements within a time range or computing aggregate functions. Latency may vary depending on whether queries target recent or historical data.
- Storage efficiency - evaluates the efficiency of a database in compressing and storing time series data. Compression ratios and storage overhead influence long-term operational costs, especially in large-scale deployments.
- Scalability - refers to the way the system behaves as workload intensity or data volume increases. It includes both vertical scaling and horizontal scaling.
- Resource utilization metrics – These metrics, including CPU usage,

memory consumption, and disk I/O behavior, provide additional insight into system efficiency. High resource utilization under moderate workloads may indicate architectural inefficiencies or tuning limitations.

Taipalus [25] argue that evaluation should consider trade-offs rather than focusing on a single metric. A system with superior ingestion throughput may exhibit higher query latency or increased storage overhead. Therefore, comparative evaluation must interpret performance across multiple dimensions.

2.5.4 *Limitations in Existing Comparative Studies*

Although many studies have evaluated database systems under time series workloads, several limitations are evident in the literature. First, many evaluations focus on a limited subset of systems, often comparing only native time series databases while excluding relational or document-oriented alternatives. This restricts architectural diversity in analysis.

Second, some studies rely on synthetic workloads that do not accurately reflect real-world sensor environments. Simplified workloads may not capture high cardinality, bursty ingestion patterns, or long-term retention challenges. Third, configuration transparency is sometimes limited. Inconsistent tuning across systems may introduce bias, particularly when default settings vary significantly.

Fourth, cloud-native managed services such as Amazon Timestream are underrepresented in academic comparisons, despite their growing adoption in industrial IoT deployments. Finally, few comparative studies explicitly address marine or environmental monitoring contexts, where long-term data accumulation and reliability requirements differ from financial or infrastructure monitoring systems.

These limitations indicate the need for a structured and transparent comparative evaluation of heterogeneous database architectures under representative sensor workloads. The present study addresses this gap by systematically evaluating TimescaleDB, InfluxDB, MongoDB and Amazon Timestream, under controlled experimental conditions designed to reflect large-scale time series data management requirements.

CHAPTER 3

DESIGN AND ANALYSIS

This chapter discusses the experimental framework used in this research and how it will evaluate database technologies for large-scale time series data management. The previous chapter considered the theoretical foundations of time series databases, their architectures and characteristics. This chapter builds on that to translate those insights into structured and reproducible experimental design. The goal is to define the way the comparative evaluation will be done in a way that is methodically sound, transparent and aligned with the requirements of the specific context of the research.

The evaluation will focus on assessing the scalability and performance of the selected database systems under representative time series workloads. Instead of basing the research on abstract comparisons or isolated performance claims, this study adopts an experimental approach that is controlled and in which a number of database technologies are subjected to identical workloads under similar conditions. This ensures that any differences observed in the performance can be attributed to the architectural and implementation characteristics instead of inconsistencies in the setup of the experiment.

To meet this goal, the chapter defines key components of the experimental design. It establishes its evaluation framework and methodological approach. This includes defining the scope of comparison, mentioning the relevant performance dimensions and outlining the principles that are used to guarantee fairness and reproducibility. Then, it describes how different database systems were selected to be included in the study alongside their rationale. Thirdly, it details the dataset design and workload models used to simulate realistic data generated by sensor-based systems.

This chapter also specifies the benchmarking approach and the experimental environment include the configurations of the system, deployment conditions and hardware setup. These elements are necessary for making sure that performance measurements remain consistent and comparable across systems. It defines the metrics that are used to assess system performance, including query latency, ingestion throughput, storage efficiency, scalability behavior and re-

source utilization. Each metric is operationalized based on how it is measured and interpreted inside the experimental framework.

Finally, the chapter outlines the experimental procedure and discusses the potential ways the design is limited. This includes considerations related to system configuration constraints, dataset realism and generalizability of the results. These factors are addressed to make sure the research is balanced and transparent.

3.1 Design and Analysis

To evaluate database systems for time series data management, one needs a structured methodological approach that ensures comparability, consistency and reproducibility. To achieve this, this study adopts the evaluation framework proposed by Brown and Wallnau [3], which offers a systematic process for assessing competing technologies on the basis of measurable performance metrics. This framework is especially fitting for comparative system evaluation because of its emphasis on the identification of relevant performance dimensions, designing representative experiments and analyzing the observed differences between technologies. The framework is organized into three primary phases: descriptive modeling, experiment design and experiment evaluation. Every phase plays a critical and separate role in structuring the research process and making sure the evaluation stays grounded both in its theoretical understanding and empirical evidence. In the context of this study, the phases are mapped as follows:

The descriptive modeling phase sets up the relationship between the problem domain and the technologies being evaluated. It involves identifying the major characteristics of the workload and the system requirements which influence performance outcomes. As addressed in Chapter 2, the descriptive modeling showed the properties of time series data, the architectural features of database systems and the problems associated with sensor workloads. The insights from that chapter inform the selection of performance metrics and the design of the experimental workloads used in this research.

The experiment design phase is the focus of this current chapter. It translates the conceptual understanding that was developed in the descriptive modeling into a concrete plan for experimentation. This phase will define the scope for evaluation, selected database systems and design the representative workloads and datasets. It will also specify the metrics to be used to assess performance. A key objective here is to make sure all systems get evaluated under consistent conditions to enable meaningful comparison. To achieve comparability, each database is subjected to a similar dataset, similar workload patterns and a similar hardware environment. That way, differences in performance can be attributed to system behavior instead of external factors. To do this, system configuration is considered carefully, including the indexing strategies, data

ingestion methods and query execution settings. Where possible, default configurations mirror typical deployment scenarios while making sure no system is unfairly disadvantaged because of misconfiguration.

During the experimental design phase, the formulation of evaluation criteria is considered. Instead of using just one performance metric, this study adopts a multi-dimensional evaluation approach which considers different aspects of system performance. These include query latency, which measures how responsive the system is to analytical queries, ingestion throughput, which is about the system's ability to handle continuous data input and storage efficiency, which is about how effective data compression and storage utilization are. The research may also consider scalability, which is about the way performance changes with increases in data volume and resource utilization which offers insight into system efficiency under load.

Throughout the experiment design, principles of transparency and repeatability are considered. All aspects of the setup including system configurations, hardware specifications and workload definitions are explicitly documented. This makes sure that the results can be reproduced under similar conditions, which is a fundamental requirement for empirical research. Besides, experimented are conducted a number of times to account for differences in system performance and the results are averaged to offer a more reliable representation of system behavior.

The final phase of the Brown and Wallnau [3] framework, experiment evaluation, involves executing the defined experiments, collecting the data and analyzing the results. This phase will be addressed in subsequent chapters where selected database systems are evaluated based on metrics defined in this chapter. The analysis is intended to identify performance trade-offs between systems and understand how differences in architecture influence observed behavior under time series workloads. The framework adopted strengthens the validity of the findings and offers a clear basis for interpreting the results in relation to the research questions defined earlier.

3.2 Selection of Database Systems

For this study, the selection of database technologies is guided by the need to evaluate systems which represent distinct architectural approaches to time series data management while still staying practically deployable within the experimental environment scope. Instead of attempting to evaluate all possible time series databases, this study stays focused by selecting a few representative systems that allow for meaningful comparisons across different philosophies of design. Three systems are picked for the initial phase of the evaluation – MongoDB, TimescaleDB and InfluxDB. These systems were picked because together, they represent different paradigms for managing time series data – native time series database architectures, document-oriented storage and rela-

tional time-series extensions. This diversity ensures the study can examine how fundamentally different design decisions can influence system behavior under identical workload conditions.

TimescaleDB is picked to represent relational database systems extended for time series workloads. It is built on top of PostgresSQL so it retains its relational model and SQL interface while adding time-based partitioning and optimizations for performance that are tailored to temporal data. It is included to allow the study to find out how far a relational system can adapt to handle time-stamped data without abandoning established principles.

InfluxDB is added to the list as a purpose-built time series database intended to specifically serve high throughput ingestion and improve efficiency of time-based querying. It has a storage engine, built-in retention mechanism and query model which are optimized for temporal workloads. Investigating InfluxDB provides insight into the limitations and performance advantages of specialized systems which prioritize time series operations over general-purpose use.

MongoDB represents document-oriented NoSQL approaches to time series data management. It has a flexible schema design and very recently introduced time series collections to allow it to accommodate heterogenous sensor data with diverse structures. It has been included in the study to provide insight into how a schema-flexible, general-purpose system performs when applied to structured time series workloads.

The selection of the above systems was also influenced by the practical considerations related to integration and implementation. All three databases offer mature Python libraries including pymongo for MongoDB, psycopg2 for TimescaleDB and influxdb-client for InfluxDB. These drivers make it possible to keep the interaction patterns consistent across systems within the experimental framework. Because there are stable client libraries, performance measurements can be conducted programmatically and reproducibly.

Notably, this study is designed to be extensible. While the initial evaluation will focus on these three systems, the framework for experimentation allows for the inclusion of more databases at a later stage. This modularity is made possible by standardized dataset formats, standardized measurement procedures and standardized definitions for the workloads. Consequently, future systems can be added into the evaluation without requiring major changes to the overall design. The goal of this selection is not to find one superior database system, but to examine how different architectural approaches work under similar workload conditions. A focus on a small but representative set of systems makes sure the evaluation remains both analytically meaningful and manageable.

3.3 Dataset Design and Characteristics

How the dataset is designed in this study is critical to the experimental framework because it directly influences the relevance and validity of the study. Instead of relying on pre-existing benchmark datasets, this study creates a synthetic dataset that mirrors the characteristics of time series data from sensor deployments. That way, the study has precise control over data properties like structure, distribution and volume, making sure that the dataset aligns with the operational context defined earlier.

The dataset is made to simulate data generated by a sensor network similar to the Smart Ocean platform. Every record in the set represents a time-stamped record produced by a sensor devise. Each record is structured to include a sensor identifier, timestamp field and many measurement attributes that correspond to operational or environmental variables. These attributes could include values like salinity, temperature, pressure or other sensor-specific metrics. This is done to reflect the multidimensional nature or data from real-world sensors.

To support comparative evaluation across the different database systems, the dataset is defined in a way that can be mapped consistently to each model. For relational systems like TimescaleDB, the dataset is saved as a structured table with columns that are clearly defined. For InfluxDB, the same data is mapped to tags, measurements and fields with the different sensor identifiers represented as tags to support efficiency in filtering. In MongoDB, the records get stored like documents with field structures that are flexible to allow measurement attributes to get nested inside each observation.

Datasets are programmatically generated to make sure they are reproducible and scalable. Synthetic data will give the study control over key parameters such as the number of sensors, how frequently data is generated and the duration of the simulation observation period. The idea is to be able to adjust these parameters to produce datasets of different sizes and enable the evaluation of system behavior under varying data volumes. For instance, increasing the number of sensors or sampling frequency creates higher ingestion rates while stretching the time horizon raises the total dataset size.

To make sure that experiments are consistent, the dataset is generated once and then reused across different systems. This removes variability that is introduced by differences in data content and makes sure each system is evaluated under identical conditions. The process of data generation is documented fully including the scripts used and parameters applied to enable reproducibility.

3.4 Workload Design

This workload design matters to the validity of the evaluation because it defines the operations through which database systems get compared and exercised. While the dataset informs the structure and content of the stored data, the work-

load specifies the way the data is written, accessed and analyzed. Within the context of time series systems, workload design has to reflect both continuous ingestion of new data and query execution which retrieves and processes historical data. This study's workload is made to represent two primary classes of operations – query workloads and ingestion workloads. These categories capture the dominant usage patterns in time series applications.

3.4.1 *Ingestion Workload*

The ingestion workload simulates how sensor-generated data arrives. Data is entered into each database system in sequential batches to reflect the nature of time series data. Instead of entering records individually, the study batches insertion strategies so as to approximate realistic data pipelines where data gets transmitted in grouped intervals instead of as isolated entries. Batch size gets treated like a configurable parameter, letting the evaluation examine how different systems respond to different ingestion patterns.

To make sure that operations are fair across systems, ingestion operations get implemented with the respective database drivers in Python. MongoDB interactions get handled through pymongo, TimescaleDB uses psycopg2 and InfluxDB is accessed via its Python client library. The standardized interaction layer makes sure that all systems get subjected to comparable application-level behavior. Ingestion performance gets measured in terms of throughput, set as the number of records that get successfully inserted per unit time.

3.4.2 *Query Workload*

Besides ingestion, workload includes queries that are designed to evaluate how the system performs under normal monitoring and analytical scenarios. The queries are made to reflect common operations done on time series data.The query workload is divided into categories based on how they are intended to function. Time-range retrieval is where the system retrieves all observations inside of a specified interval for a given set of sensors or sensor. These queries test how able the system is to scan and filter time-stamped data.

The second category deals with aggregation queries where statistical summaries like counts, minimums, averages and maximum values get computed over a set time window. These queries are necessary for reporting and monitoring applications and need efficient processing of large data volumes. A third category includes multi-dimensional filtering queries where data gets retrieved based on a combination of metadata attributes and time constraints. These queries typically add a layer of complexity because they require the system to evaluate many filtering conditions at the same time.

Query workload is executed after there is sufficient volumes of ingested data to make sure that the measurements for performance reflect realistic dataset sizes. The queries are repeated a number of times to account for differences

in execution time and the results get averaged to produce stable performance estimates.

3.4.3 *Query Execution Approaches*

This study uses two complementary approaches to provide a comprehensive evaluation of query performance – native query execution and driver-based execution.

3.4.4 *Workload Consistency and Control*

For workload design to be effective, it must also factor in consistency across all evaluated systems. For this, the same sequence of operations will be used in each database with equivalent query definitions and identical datasets. Differences in the query syntax between systems get resolved via careful mapping to make sure that each query runs the same logical operation even if the implementation underneath it is different.

Workload execution is controlled so that there is no interference between query operations and ingestion unless that is explicitly required. Query workloads and ingestion are executed separately to isolate their performance characteristics. This makes sure that the measurements do not get affected by concurrent interactions between workloads. The study is performed this way to make sure the basis for the evaluation is consistent and realistic.

3.5 Experimental Environment and System Configuration

The environment of the experiment refers to the set of conditions under which all the database systems get deployed and evaluated. When the environment is well defined, it makes performance comparisons reproducible, fair and attributable to characteristics of the system instead of external factors. This section describes the software configuration, hardware setup and deployment approach that is used in the study.

3.5.1 *Hardware Environment*

All experiments are run inside a controlled hardware environment to get rid of variability from differences in computational resources. The evaluation is done on a single-node system set with sufficient memory, power and storage capacity to support the defined workloads. This is a deliberate design choice intended to isolate the intrinsic performance characteristics of each database system. Distributed deployments may be common in large-scale production environments but introducing multiple nodes would add a layer of complexity related to synchronization, network communication and cluster management. The focus on a single node configuration makes it possible for the study to ensure that performance differences reflect the efficiency of each system's storage

engine, query processing mechanism and indexing strategy. Hardware specifications including memory capacity, CPU type, storage type and number of cores are documented to make the evaluation reproducible. Where possible, storage devices that have consistent performance characteristics get used to minimize variability in disk I/O behavior.

3.5.2 *Software Environment*

Each system is deployed with stable and widely adopted versions to make sure the evaluation is reliable and consistent. The software environment includes the Operating System, database engines and the client libraries used for interaction. InfluxDB, MongoDB and TimescaleDB are installed and configured accordingly. The default configurations become the baseline with minimal adjustments made only according to need to make sure there is the time series functionality or ensure compatibility. That way, the experiment reflects realistic deployment scenarios where systems often get used with standard configurations instead of tuned setups. The experimental framework is run in Python which works like the orchestration layer for query execution, ingestion and data generation. Python is chosen because it has an extensive ecosystem of database drivers and it is suitable for automation and scripting. A single programming language across all systems makes sure that there is operational consistency.

3.5.3 *Database Configuration*

Even though the default configurations are utilized as the baseline, certain database-specific settings get adjusted to align with the study requirements. For instance, TimescaleDB is configured to enable time-partitioned tables and make sure that data is organized in a way that is consistent with how it will be used. InfluxDB gets configured with necessary measurement definitions and MogoDB uses time series collections to optimize query performance and storage. Indexing strategies get defined consistently across systems to support query workload. In the framework, time-based indexes are also created to enable efficient by timestamp filtering while extra indexes may be applied to sensor identifiers or other attributes that are frequently queried. Care is taken to make sure that no system is unfairly advantaged or disadvantaged and that similar logical structures get used across databases.

3.5.4 *Execution Control and Isolation*

To make sure there is consistency, experiments are run under conditions which minimize external process interference. Background applications are limited during testing and the resources of the system monitored to make sure there is no unexpected load that can affect performance measurements. Each experiment is run independently and the database system is reinitialized or reset between runs according to need. This is to prevent caching effects or residual data from influencing the next experiment. Where the experiment requires

caching behavior, it is explicitly accounted for in the experimental design.

3.5.5 Considerations for Reproducibility

Reproducibility is necessary in any empirical evaluation. To support it, all aspects of the experimental environment are recorded including software versions, hardware specifications, execution procedures and configuration settings. The scripts that are used for ingestion, data generation and query execution are preserved and may be reused to replicate the experiments.

3.6 Evaluation Metrics

This study evaluates database systems using a set of carefully defined performance metrics that capture different aspects of system behavior for time series workloads. Instead of relying on just one measure of performance, the study uses several benchmarks which reflect different operational requirements of large-scale sensor data systems. Every metric is defined the way it operates, with clear procedures for measurement to ensure comparability and consistency across all the studied database technologies.

3.6.1 Ingestion Throughput

This metric considers the rate at which the system can accept and persist incoming data. It is a measure of the number of records that are successfully inserted into the database per unit time, often expressed in records per second. It is a necessary metric for time series systems where continuous data streams have to be ingested with minimal delay or data loss. To measure it, data is added into each database in controlled batches and the total time used to complete the insertion gets recorded. Throughput gets calculated as the ratio of the number of inserted records to the total time used. The measurements are taken after an initial warm up to make sure that the results reflect a steady state system rather than initialization overhead.

3.6.2 Query Latency

Query latency is a measure of the time it takes for a database system to process a query and return results. It accounts for the system's response time for analytical and monitoring activities, which is a critical factor for applications that depend on the availability of data in a timely manner. Latency is determined by a timing method that measures the beginning and end of a query execution process. For a driver-based execution, the beginning time is measured immediately before the query execution, and the end time is measured after the results are received by the application. The difference between these times is the query latency, which includes the client and network times.

3.6.3 Storage Efficiency

Storage efficiency measures how well a database system uses storage space in handling its time series data. It is defined as the ratio of physical storage space used by the database system for its data to the input data set's size. In calculating storage efficiency, the input data set's storage space is obtained after the ingestion process is complete, including all data structures maintained by the database system, which include indexes and metadata. The input data set's size, on the other hand, is obtained before the ingestion process, allowing for a direct comparison of the input data set and the stored data space. This is a critical factor in a database system, especially in long-term usage, as data is constantly being added over time. A database system that has a higher storage efficiency is ideal for usage in a storage space constraint or a long-term retention scenario.

3.6.4 Scalability

Scalability is a measure used to determine the changes in system performance as the amount of data or intensity of work is increased. Scalability is not defined by a specific numerical value, but by a trend in system performance over different amounts of data. In this particular study, the concept of scalability is analyzed by increasing the size of the data set used in the experiment, repeating the ingestion and query evaluation processes, and analyzing the results obtained. This way, the study can determine the trend in the data ingestion throughput of the system. A system is defined to be scalable if its performance is stable or decreases gracefully in response to increased workload intensity. A sudden decrease in system performance may imply a problem in storage structures, indexing, or resource management.

3.6.5 Resource Utilization

Resource utilization offers an insight into the efficiency of a database system with regards to its ability to effectively utilize resources. The metrics include CPU usage, memory, and disk I/O. Resource utilization is tracked for both ingestion and query execution. High resource usage can imply poor design or configuration of a system, while low resource usage with high performance can imply effective optimization. Resource utilization is not a key aspect of the study but offers useful information for understanding performance results. For instance, two systems can have similar performance but drastically different resource requirements.

3.7 Data Analysis Methods

The analysis of the results of the experiment needs a structured approach that guarantees reliability, consistency and meaningful interpretation of the observations. This section details the methods used to process, aggregated and compare

the data that is collected in the evaluation. The goal is to turn the raw observations into interpretable results while lowering the influence of measurement noise and variability.

3.7.1 Repetition and Averaging

To make the measurements more reliable, each experiment is performed multiple times in the same environment. This reduces the impact of the system's transient states and the presence of background processes, among other sources of indeterminism. The results from each run are combined using the average function. This gives a more reliable view of the system's performance than using a single run's result. In addition, the range of the results is used to determine the consistency of the system's performance.

3.7.2 Normalization of Results

To make the results more comparable, some of the results, such as ingestion throughput, are normalized to a standard scale. This is done, for instance, by normalizing the ingestion throughput relative to the best system's throughput. This is important because it makes it easy to compare the performance of systems whose performance is measured in different ranges.

3.7.3 Comparative Analysis

The main analysis technique is the comparison of performance metrics between various database systems. For instance, for a given performance metric, results are grouped into comparative tables and graphical displays, making it easy to spot performance differences between various database systems. Comparative analysis is done along various axes, including data size, workload types, and complexity. By conducting a multidimensional analysis, it is possible to spot trends in database system behavior, for example, how performance changes with increasing data sizes.

3.7.4 Trend Analysis

Trend analysis is another analysis technique used to analyze performance metrics against changing workload conditions. For example, ingestion throughput can be plotted against data size to understand performance changes with time. Similarly, query latency can be analyzed against different types of queries or time intervals. Understanding trends in performance metrics, makes it possible to understand the performance and reliability of various database systems. For instance, a constant trend in performance metrics is a guarantee of a reliable database system.

3.7.5 Handling Variability and Outliers

It is possible that performance measurements occasionally have outliers resulting from system interruptions, measurement anomalies or resource contention. These outliers get identified through inspection of statistical summaries and repeated measurements. Where possible, the outliers are excluded from the analysis to avoid a distortion of the results. However, the exclusion is carefully justified to make the process transparent. In cases where that variability is significant, it gets reported alongside average values to give a complete picture of system behavior.

3.7.6 Experimental Procedure

The experimental procedure outlines the steps taken to carry out the evaluation process. The study's formalization of this process guarantees that all database systems are subject to the same environment, hence a consistent performance measurement process. The process begins with the initialization of the experimental environment. The database systems are set up, initialized, and verified to ensure correct functioning before the experiment. The database systems are reset to a clean state before running the experiment, where necessary, to avoid any effects from previous runs of the experiment.

After initialization, the data set is fed into the database system through a predefined workload. The insertion of data into the database occurs in batches, with the process's timing being recorded. The process begins with a small warm-up phase to allow the database system to settle before the performance measurement process begins. The initialization process of the database systems is avoided during the experiment to ensure a fair evaluation of the database systems' performance.

3.8 Threats to Validity and Design Limitations

The experimental design outlined in this chapter aims to make the experimentation fair and the experimentation environment controlled. However, there are some limitations and potential threats to validity which must be acknowledged. Identifying the limitations is essential to the interpretation of the results accurately and to coming to useful conclusions.

One potential threat to internal validity comes from any necessary differences in system configuration. Even though efforts are made to make all the configurations standard across database systems, each system is unique and has different tuning parameters and optimization strategies. Going with the default or adjusting configurations minimally may not fully reflect the optimal performance for each system. At the same time, extensive tuning could bring bias into the equation if the tuning is not done consistently. The study mitigates this risk by using a uniform configuration strategy and documenting all used settings.

Another limitation pertains to the use of synthetic data. Synthetic datasets are preferrable because they allow precise control over the characteristics of the data but they may fail to capture all the complexities of real-world sensor data like missing values, irregular sampling patterns and unpredictable data bursts. Because of this performance observed under the experiment conditions may differ from behavior in real world applications. However, the design of the dataset is set to approximate the key characteristics of time series workloads so that the basis of evaluation is reasonable.

While there is a rationale for the use of a single-node experimental environment, it introduces extra limitations. The setup allows for controlled comparison of the way the systems behave, but it does not capture the complexities of distributed deployments in which factors like replication, network latency and load balancing play a major role. This may mean that the results won't fully generalize to large-scale distributed systems.

Another consideration for the research is measurement accuracy. Timing measurements obtained via application-level instrumentation could include overhead from network communication or client-side processing. Even though this reflects realistic scenarios for use, it may not isolate the internal performance of the database engine. Including native query execution measurements helps to lower this risk by offering an alternative perspective on the system's performance.

The selection of database systems may also limit external validity. This study focuses on a subset of the available technologies and the results may not map onto systems that are not included in the evaluation. However, deliberate effort has been made to select databases that represent diverse architectural approaches to allow for meaningful insights into different design paradigms.

Finally, workload design choices may affect how performance is observed. The workload has been constructed to reflect typical time series operations but variations in ingestion strategies or query patterns could produce different results. This study addresses this by defining the workloads clearly and then applying the consistently across different runs and systems.

3.9 Synthesis of Design Decisions

The experimental framework design presented in this chapter reflects an intentional effort to balance methodological rigor with practicality. Each component of the design from workload constriction to database selection and performance measurement, is informed by the need to enable fair and meaningful comparison of database systems under time series workloads.

One central design decision is running the experiments in controlled environments where all systems get evaluated under identical conditions. This is done to make sure that performance differences are directly attributable to system be-

havior instead of external variability. Using a single-node configuration helps to anchor this objective by isolating the characteristics intrinsic to each database system even though it adds limitations to distributed scalability analysis.

How the database systems are selected reflects an emphasis on architectural diversity. The study includes relational, document-oriented approaches and native time series to examine the way different design philosophies influence performance. This diversity improves the analytical value of the study. At the same time, the workload and dataset designs are structured to reflect realistic usage scenarios while guaranteeing control over experimental variables. The use of synthetic data provides flexibility and reproducibility and defining query and ingestion workloads ensures that each system is evaluated under representative conditions. Separating query and ingestion phases makes clear analysis possible for each performance dimension without any interferences.

Performance metrics are defined in ways that capture many aspects of system behavior including scalability, throughput, latency, storage efficiency and resource utilization. This approach acknowledges that no one metric is enough to fully characterize system performance and allows for the comparison to be more nuanced.

The experimental procedure and the methods for data analysis are meant to ensure consistency, transparency and repeatability. Repeated measurements, systematic analysis techniques and structured data collection contribute to reliable results and support the validity of the conclusions this study draws. Taken together, these decisions for design create a coherent and robust framework to evaluate database systems for time series data management. The framework also provides a basis for the experimental results provided in subsequent chapters.

CHAPTER 4

IMPLEMENTATION AND PROTOTYPES

```
public static void main(String[] args) {

  int b, h, d;
  String btext, htext, dtext;

  [ ... ]

  int volum = b * h * d;

  String respons =
      "Volum [" + htext + "," + btext + "," + dtext + "] = " + volum;

}
```

CHAPTER 5

EVALUATION AND RESULTS

Table 5.1 gives an example of how to create a table.

Config	Property	States	Edges	Peak	E-Time	C-Time	T-Time
22-2	A	7,944	22,419	6.6 %	7 ms	42.9%	485.7%
22-2	A	7,944	22,419	6.6 %	7 ms	42.9%	471.4%
30-2	B	14,672	41,611	4.9 %	14 ms	42.9%	464.3%
30-2	C	14,672	41,611	4.9 %	15 ms	40.0%	420.0%
10-3	D	24,052	98,671	19.8 %	35 ms	31.4%	285.7%
10-3	E	24,052	98,671	19.8 %	35 ms	34.3%	308.6%

Table 5.1: Selected experimental results on the communication protocol example.

CHAPTER 6

CONCLUSIONS AND FUTURE WORK

6.1 Main Contributions

6.2 Conclusions on Research Questions

6.3 Threats to Validity

6.4 Future Work

BIBLIOGRAPHY

[1] Daniel Abadi, Peter Boncz, Stavros Harizopoulos, Stratos Idreos, and Samuel Madden. The design and implementation of modern column-oriented database systems. *Foundations and Trends in Databases*, 5(3): 197–280, 12 2013. ISSN 1931-7883. doi: 10.1561/1900000024. URL https://doi.org/10.1561/1900000024. 2.4.2

[2] Debasis Bandyopadhyay and Jaydip Sen. Internet of things: Applications and challenges in technology and standardization. *Wireless Personal Communications*, 58(1):49–69, 2011. doi: 10.1007/s11277-011-0288-5. URL https://doi.org/10.1007/s11277-011-0288-5. 2.1.2

[3] Alan Brown and Kurt Wallnau. Framework for evaluating software technology. *Software, IEEE*, 13:39 – 49, 10 1996. doi: 10.1109/52.536457. URL https://ieeexplore.ieee.org/document/536457. 1.4, 1.1, 3.1, 6.4

[4] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM Symposium on Cloud Computing*, SoCC '10, page 143–154, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781450300360. doi: 10.1145/1807128.1807152. URL https://doi.org/10.1145/1807128.1807152. 2, 2.5.2

[5] Feliciano Pedro Francisco Domingos, Ahmad Lotfi, Isibor Kennedy Ihianle, Omprakash Kaiwartya, and Pedro Machado. Underwater communication systems and their impact on aquatic life—a survey. *Electronics*, 14(1), 2025. ISSN 2079-9292. doi: 10.3390/electronics14010007. URL https://www.mdpi.com/2079-9292/14/1/7. 1

[6] Forskningsrådet. Sfi smart ocean – flexible and cost-effective monitoring for management of a healthy and productive ocean. https://prosjektbanken.forskningsradet.no/project/FORISS/309612, n.d. Accessed: 2026-02-27. 1

[7] Michael Geurts. Time series analysis: Forecasting and control. *Journal of Marketing Research*, 14(2):269–269, 1977. ISSN 00222437. URL http://www.jstor.org/stable/3150485. 2, 2.1.1

[8] Graphite Project. Overview. Graphite Documentation. URL https://graphite.readthedocs.io/en/latest/overview.html. Accessed: 16 March 2026. 2.2.5

[9] Jim Gray, editor. *The Benchmark Handbook: For Database and Transaction Processing Systems*. Morgan Kaufmann, San Mateo, CA, 1992. URL http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&

local_base=BVB01&doc_number=005950188&sequence=000001&line_number=0001&func_code=DB_RECORDS&service_type=MEDIA. 2.5.1

[10] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660, 2013. ISSN 0167-739X. doi: https://doi.org/10.1016/j.future.2013.01.010. URL https://www.sciencedirect.com/science/article/pii/S0167739X13000241. Including Special sections: Cyber-enabled Distributed Computing for Ubiquitous Cloud and Network Services and Cloud Computing and Scientific Applications — Big Data, Scalable Analytics, and Beyond. 2, 2.1.2, 2.1.3, 2.4.1

[11] InfluxData, Inc. In-memory indexing and the time-structured merge tree (tsm). InfluxDB OSS v1 Documentation. URL https://docs.influxdata.com/influxdb/v1/concepts/storage_engine/. Accessed: 16 March 2026. 2.2.2

[12] Søren Kejser Jensen, Torben Bach Pedersen, and Christian Thomsen. Time series management systems: A survey. *CoRR*, abs/1710.01077, 2017. URL http://arxiv.org/abs/1710.01077. 1, 1.1.2, 2.3

[13] Andrew Lamb, Matt Fuller, Ramakrishna Varadarajan, Nga Tran, Ben Vandiver, Lyric Doshi, and Chuck Bear. The vertica analytic database: C-store 7 years later. *Proc. VLDB Endow.*, 5(12):1790–1801, August 2012. ISSN 2150-8097. doi: 10.14778/2367502.2367518. URL https://doi.org/10.14778/2367502.2367518. 2.2.1

[14] Janot Mendler de Suarez, Biliana Cicin-Sain, Kateryna Wowk, Rolph Payet, and Ove Hoegh-Guldberg. Ensuring survival: Oceans, climate and security. *Ocean and Coastal Management*, 90:27–37, 2014. ISSN 0964-5691. doi: https://doi.org/10.1016/j.ocecoaman.2013.08.007. URL https://www.sciencedirect.com/science/article/pii/S0964569113001877. 1

[15] MongoDB Documentation Team. Time series collections. https://www.mongodb.com/docs/manual/core/timeseries-collections/, n.d. Accessed: 2026-03-06. 2.2.4

[16] MongoDB, Inc. Documents. MongoDB Manual. URL https://www.mongodb.com/docs/manual/core/document/. Accessed: 16 March 2026. 2.2.4

[17] Mohammad Nasar and Mohammad Abu Kausar. Suitability of influxdb database for iot applications. *International Journal of Innovative Technology and Exploring Engineering*, 8(10):1850–1857, August 2019. doi: 10.35940/ijitee.j9225.0881019. URL http://dx.doi.org/10.35940/ijitee.J9225.0881019. 2.2.2

[18] Tuomas Pelkonen, Scott Franklin, Justin Teller, Paul Cavallaro, Qi Huang,

Justin Meza, and Kaushik Veeraraghavan. Gorilla: a fast, scalable, in-memory time series database. *Proc. VLDB Endow.*, 8(12):1816–1827, August 2015. ISSN 2150-8097. doi: 10.14778/2824032.2824078. URL https://doi.org/10.14778/2824032.2824078. 2.1, 2.4.4, 6.4

[19] Prometheus Authors. Storage. Prometheus Documentation. URL https://prometheus.io/docs/prometheus/latest/storage/. Accessed: 16 March 2026. 2.2.5

[20] Prometheus Overview. Prometheus: Overview. https://prometheus.io/docs/introduction/overview/, n.d. Accessed: 2026-03-06. 2.2.5

[21] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. Clickhouse - lightning fast analytics for everyone. *Proc. VLDB Endow.*, 17(12):3731–3744, August 2024. ISSN 2150-8097. doi: 10.14778/3685800.3685802. URL https://doi.org/10.14778/3685800.3685802. 2.2.3

[22] SFI Smart Ocean. About – sfi smart ocean, n.d. URL https://sfismartocean.no/about/. Accessed: 2026-02-28. 1.1.1

[23] Leonid B. Sokolinsky. Survey of architectures of parallel database systems. *Programming and Computer Software*, 30(6):337–346, 2004. ISSN 1608-3261. doi: 10.1023/B:PACS.0000049511.71586.e0. URL https://doi.org/10.1023/B:PACS.0000049511.71586.e0. 2.3

[24] Michael Stonebraker, Uundefinedur Çetintemel, and Stan Zdonik. The 8 requirements of real-time stream processing. *SIGMOD Rec.*, 34(4):42–47, December 2005. ISSN 0163-5808. doi: 10.1145/1107499.1107504. URL https://doi.org/10.1145/1107499.1107504. 2.1.1, 2.1.3, 2.1.4, 2.4.1, 2.4.2, 2.4.5

[25] Toni Taipalus. Database management system performance comparisons: A systematic literature review. *Journal of Systems and Software*, 208:111872, February 2024. ISSN 0164-1212. doi: 10.1016/j.jss.2023.111872. URL http://dx.doi.org/10.1016/j.jss.2023.111872. 2.2.4, 2.5.1, 2.5.3

[26] Timescale, Inc. Timescaledb, 2017. URL https://github.com/timescale/timescaledb. Accessed: 16 March 2026. 2.2.1

[27] Andrea Zanella, Nicola Bui, Angelo Castellani, Lorenzo Vangelista, and Michele Zorzi. Internet of things for smart cities. *IEEE Internet of Things Journal*, 1(1):22–32, 2014. doi: 10.1109/JIOT.2014.2306328. URL https://ieeexplore.ieee.org/document/6740844. 2.1.2, 2.1.3

APPENDIX A

SOURCE CODE

Where to find the source code . . . (if applicable)

APPENDIX B

RESEARCH DATA

Where to find the research data . . . (if applicable)

List of Figures

1.1 Software technology evaluation framework (adapted from Brown and Wallnau [3]). 14

2.1 Architecture of the Gorilla time-series monitoring database illustrating ingestion buffers and segmented storage structures used for efficient processing of temporal data. Adapted from (Pelkonen et al. [18]). 28

List of Tables

5.1 Selected experimental results on the communication protocol example. 49